A Live Demonstration of Shor's Algorithm

The Fourier Transform: Part 1

Ryan O'Farrell

University of Chicago

January 24, 2025

Table of Contents

This week (`https://arxiv.org/pdf/1301.7007`)
(*Pretending to factor large numbers on a quantum computer*)

- Mathematical basis of the DFT
- The Quantum Fourier Transform
- Application to Shor's Algorithm
- Live demonstration of Shor's!

Next week (`https://arxiv.org/pdf/2408.00075`)
(*Highly-efficient quantum Fourier transformations for certain nonabelian groups*)

- Introduction to group representations
- The non-abelian Fourier transform
- New quantum algorithms

Fourier analysis for electrical engineers

Definition

Given a *periodic* function on $\mathbb{R}$, its *Fourier series* is a set of coefficients $c_n \in \mathbb{C}$ satisfying *Parseval's identity*.

$$\frac{1}{2\pi}\int_{-\pi}^{\pi} |f(x)|^2 dx = \sum_n |c_n|^2 \qquad \text{for } f \text{ with period } 2\pi$$

Definition

Given a function on $\mathbb{R}$, its *Fourier transform* is another function on $\mathbb{R}$ satisfying *Plancherel's theorem*.

$$\int_{-\infty}^{\infty} |f(x)|^2 dx = \int_{-\infty}^{\infty} |\hat{f}(x)|^2 dx$$

What is this connection?

We can treat periodic functions as functions on S^1 and the corresponding Fourier coefficents as functions on $\mathbb{Z}$.

Question

What is this connection

$$S^1 \leftrightarrow \mathbb{Z} \qquad \mathbb{R} \leftrightarrow \mathbb{R}$$

that lets us define the Fourier transform, and how can we extend it to other groups?

Pontryagin duality

Definition

An *abelian group* is a set G equipped with an operation $+ : G \times G \to G$ satisfying

- The existence of 0 and inverses $-g \in G$ such that $g + (-g) = 0$
- Associativity: $a + (b + c) = (a + b) + c$
- Abelian: $a + b = b + a$

S^1, $\mathbb{Z}$, and $\mathbb{R}$ are all abelian groups.

Definition

Given a (locally compact) abelian group, its *Pontryagin dual* $\hat{G}$ is the group $\mathrm{Hom}(G, S^1)$ of continuous addition-preserving maps between G and the group of rotations of a circle S^1.

Computing the Pontryagin Dual

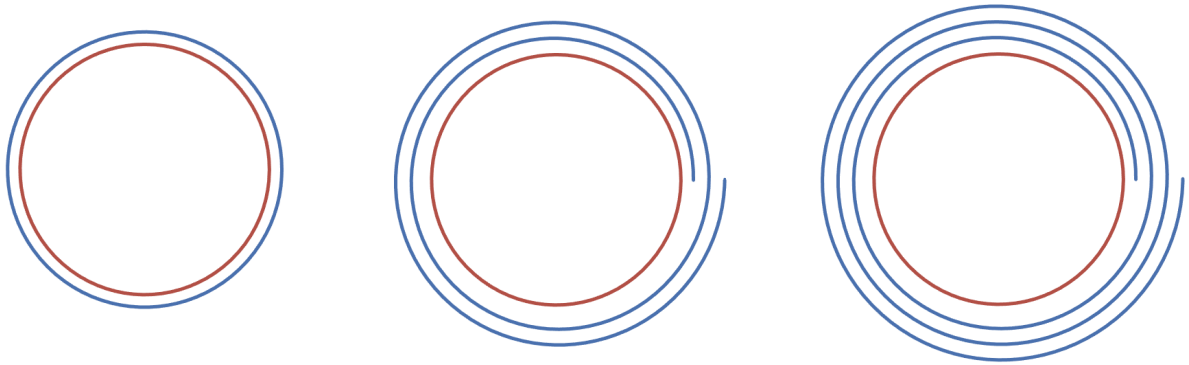


Figure: The dual of S^1 is $\mathbb{Z}$

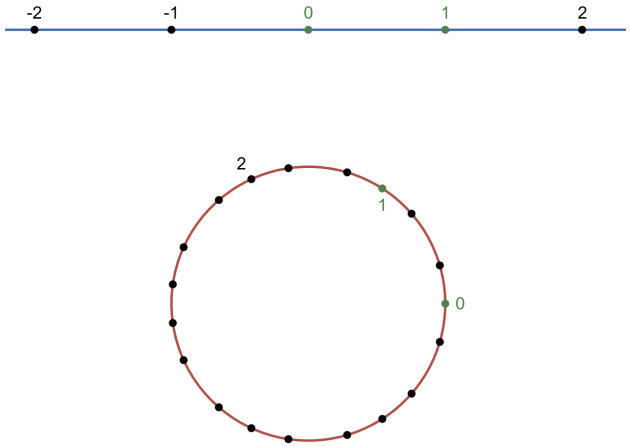


Figure: The dual of $\mathbb{Z}$ is S^1

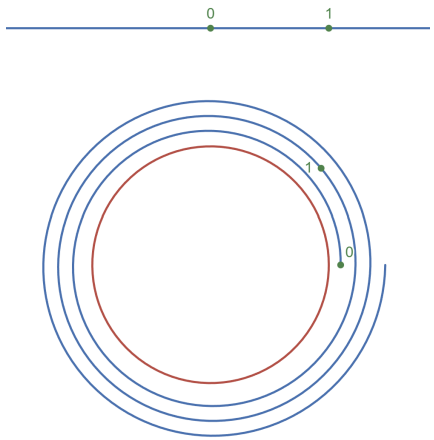


Figure: The dual of $\mathbb{R}$ is $\mathbb{R}$

The Almost-General Fourier Transform

Definition

The *Fourier transform* is a linear map from functions on an abelian group to functions on the Pontryagin dual group that satisfies a generalized Plancherel identity (norm-preserving).

$$\int_G |f|^2 dg = \int_{\hat{G}} |\hat{f}|^2 d\hat{g}$$

Using this, we can build the Fourier transform for other abelian groups!

Introducing $\mathbb{Z}/n\mathbb{Z}$

We can turn any discrete set of n points (samples) into the abelian group $\mathbb{Z}/n\mathbb{Z}$.

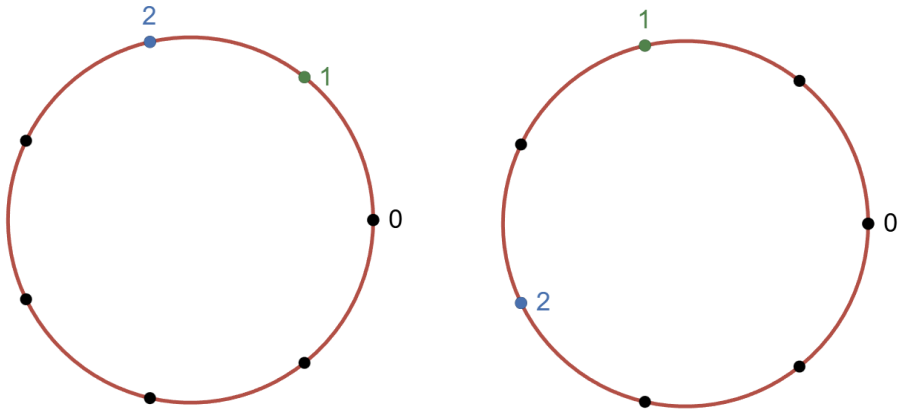


Figure: The dual of $\mathbb{Z}/n\mathbb{Z}$ is itself.

By Plancherel, the Fourier transform is an *isometry* from functions on $\mathbb{Z}/n\mathbb{Z}$ to functions on $\mathbb{Z}/n\mathbb{Z}$

The (Actual) Discrete Fourier Transform

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-2\pi i \frac{kn}{N}}$$

Or (since it's linear) as a matrix

$$W = \frac{1}{\sqrt{N}} \begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \cdots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \cdots & \omega^{2(N-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \cdots & \omega^{3(N-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \omega^{3(N-1)} & \cdots & \omega^{(N-1)(N-1)} \end{bmatrix}$$

It satisfies

$$\sum_{n=0}^{N-1} |x_n|^2 = \frac{1}{N} \sum_{n=0}^{N-1} |X_n|^2$$

The Quantum Fourier Transform

The point: Norm-preserving $\implies$ DFT is unitary!

Definition

The QFT is an algorithm for doing the discrete Fourier transform on state vectors $|\psi\rangle$, viewed as $\mathbb{C}$-valued functions on the computational basis.

Example

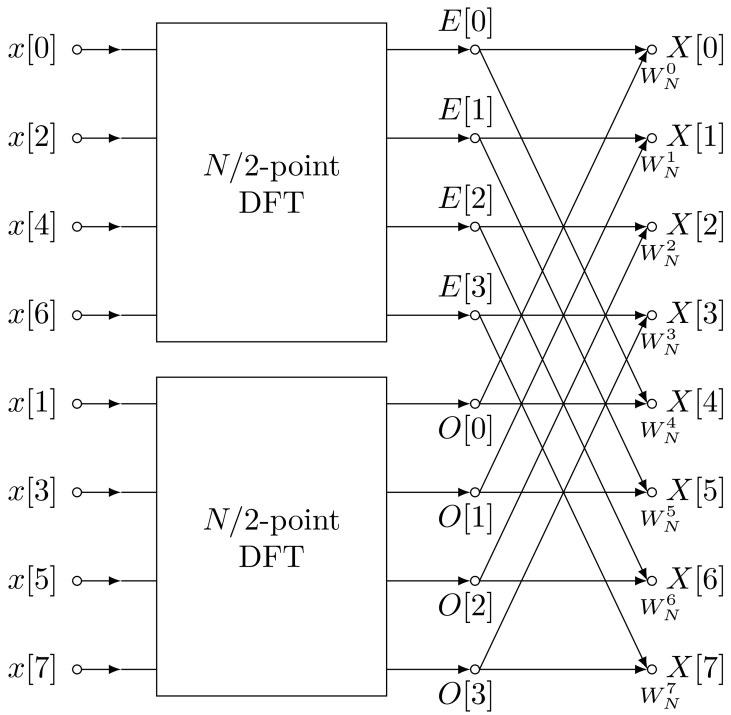
$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

is a 1-qubit QFT.

Complexity

Basic	FFT	QFT
$O(n^2)$	$O(n\log(n))$	$O(\log(n)^2)$

Both FFT and QFT exploit the subgroup structure of $\mathbb{Z}/2^n\mathbb{Z}$



Qubit Recycling

We can use the subgroup structure to read the qubits one at a time.

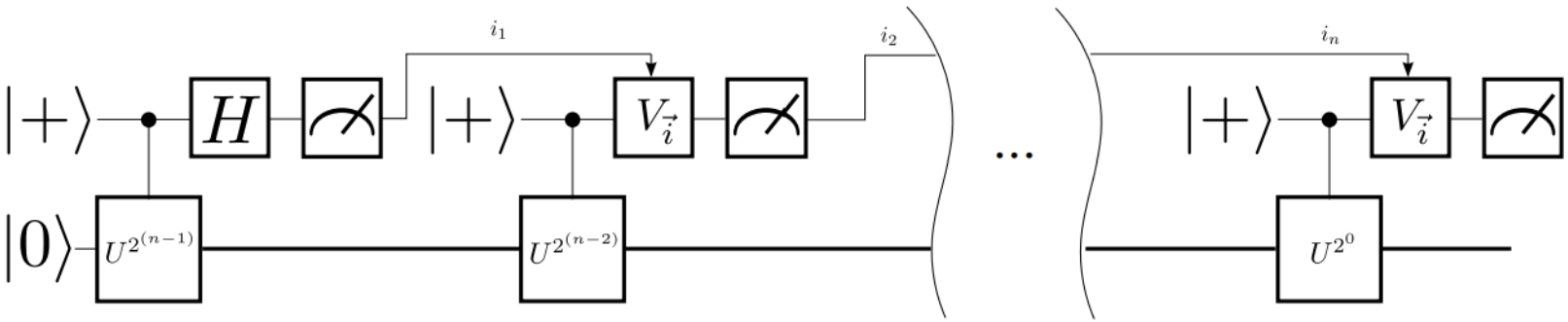


Figure: Implementation in the paper

Equivalent to QFT + measurement.

Shor's Algorithm

Given $N = pq$, Shor's algorithm finds p and q

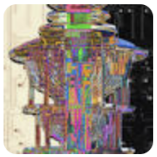
- Classically hard
- Responsible for all the "QC breaks encryption" headlines

WIRED

See How Much Faster a Quantum Computer Will Crack Encryption

It would take eight hours for a quantum computer with 20-million noisy qubits to factor RSA-2048 encryption.

Jun 4, 2025



NS New Scientist

Breaking encryption with a quantum computer just got 20 times easier

Quantum computers could crack a common data encryption technique once they have a million qubits, or quantum bits. While this is still well beyond the...

May 23, 2025



Spiceworks

Quantum computers will crack your encryption. Now what?

They're not powerful enough to break encryption yet, but they're scaling up fast. The Global Risk Institute estimates that a cryptographically relevant quantum...

Nov 10, 2025



Choosing a

1. Choose (at random) an integer $0 < a < N$.
2. Compute the greatest common divisor (GCD) of a and N. This can be found efficiently using the Euclidian algorithm[8]. If it is not 1, then $\mathrm{GCD}(a, N)$ is a nontrivial factor of N. Otherwise go on to the next step.

Important point: We're free to choose any a.

The Big Picture

We know there's some r such that $a^r \equiv 1 \pmod{N}$, so N divides $a^r - 1 = (a^{r/2} + 1)(a^{r/2} - 1)$.

If N doesn't divide either factor, then $\gcd(N, (a^{r/2} \pm 1)) = p$ or q

Goal

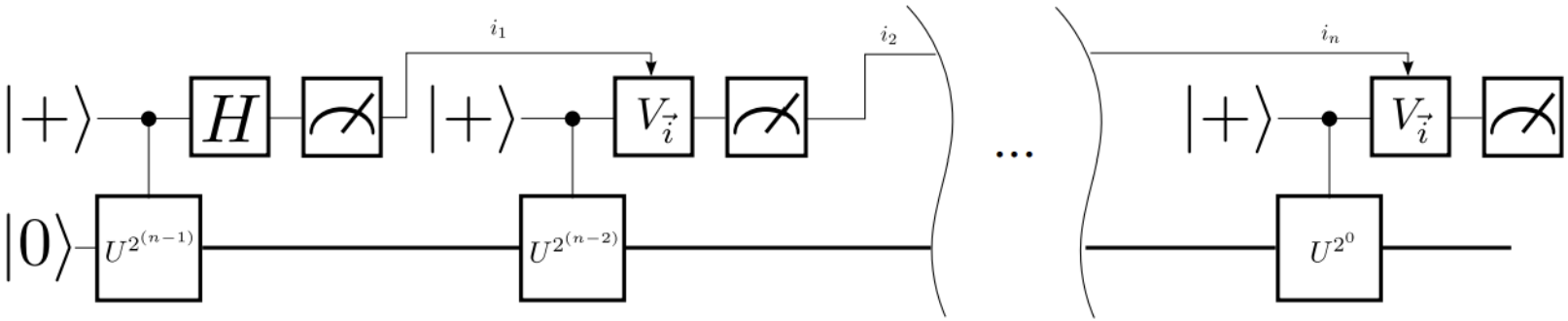
Find r.

Modular Exponentiation

3. Choose $S \equiv 2^s$ such that $N^2 \leq S < 2N^2$. Construct the quantum state

$$S^{-1/2} \sum_{x=0}^{S-1} |x\rangle|0\rangle \tag{1}$$

4. Perform a quantum computation on this state which maps $|x\rangle|0\rangle$ to $|x\rangle|a^x \mod N\rangle$. This is the slowest step, but can be done in time $O((\log N)^3)$.



(Bottom register)

QFT Measurement

5. Do the quantum Fourier transform on the first register, resulting in the state

$$S^{-1} \sum_x \sum_y e^{(2\pi i/S)xy} |y\rangle |a^x \mod N\rangle \,. \tag{2}$$

This step requires $O((\log N)^2)$ time, which is much less than the modular exponentiation of the previous step.

6. Measure the first register to obtain classical result y. With reasonable probability, the continued fraction approximation of y/S or some y'/S for some y' near y will be an integer multiple of the period r of the function $a^x \mod N$. The GCD algorithm can then efficiently find r.

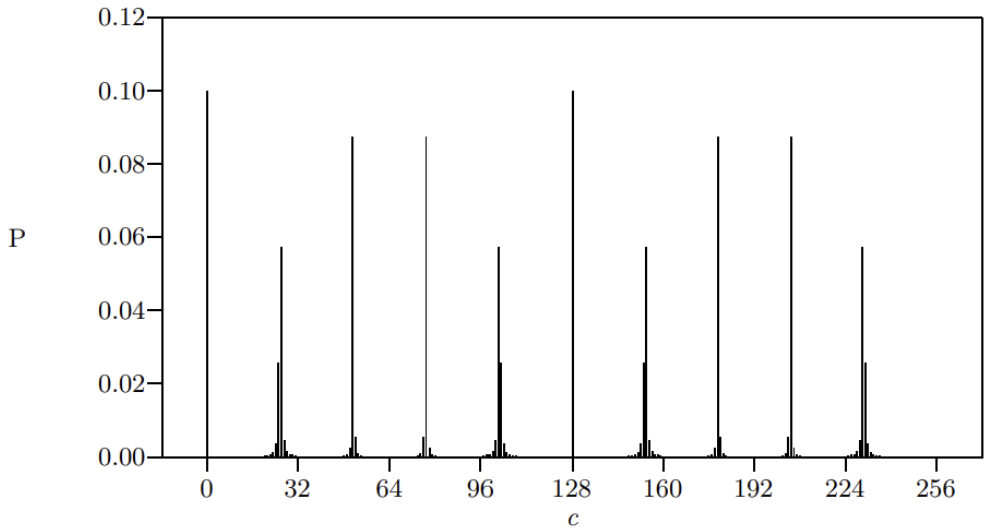


Figure: $r = 10$ (from P. Shor)

“Compiled versions”

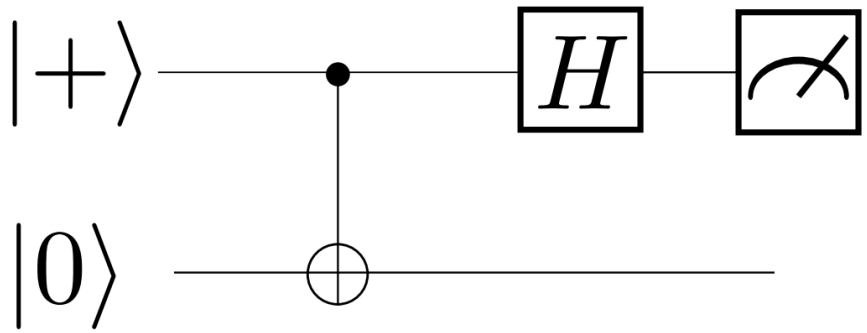
How can we choose a to make running the algorithm as easy as possible?

J. A. Smolin, G. Smith, A. Vargo

We can always choose a such that the modular exponentiation register requires only one qubit.

The Algorithm

Simplifies considerably



Measurement outcome is always 50-50 between 0 and 1 (corresponding to 0 and $\frac{1}{2}$)

Live Demonstration

We will factor $N =$
1230186684530117755130494958384962720772853569595334792197322452157264005072636575187452021997864693899564749427740638459251925573263034537315482685079170261221429134616704292143116022212404792747377940806653514195974598569021434 13

using $a =$
1029031793302493258003488818376905875264575120178567995715921117383374063780955476265714655965556097487715509708453134212472071241551710737667646125017671995537319749739035045343586527599466828935082557618400047627481255809299529939.

Running the algorithm...

Results

Incredibly, a has period 2! What a surprise!

$\gcd(a+1, N) =$
$33478071698956898786044169848212690817704794983713768568912431388982883793878002287614711652531743087737814467999489$

$\gcd(a-1, N) =$
$36746043666799590428244633799627952632279158164343087642676032283815739666511279233373417143396810270092798736308917$

These are indeed the factors of N

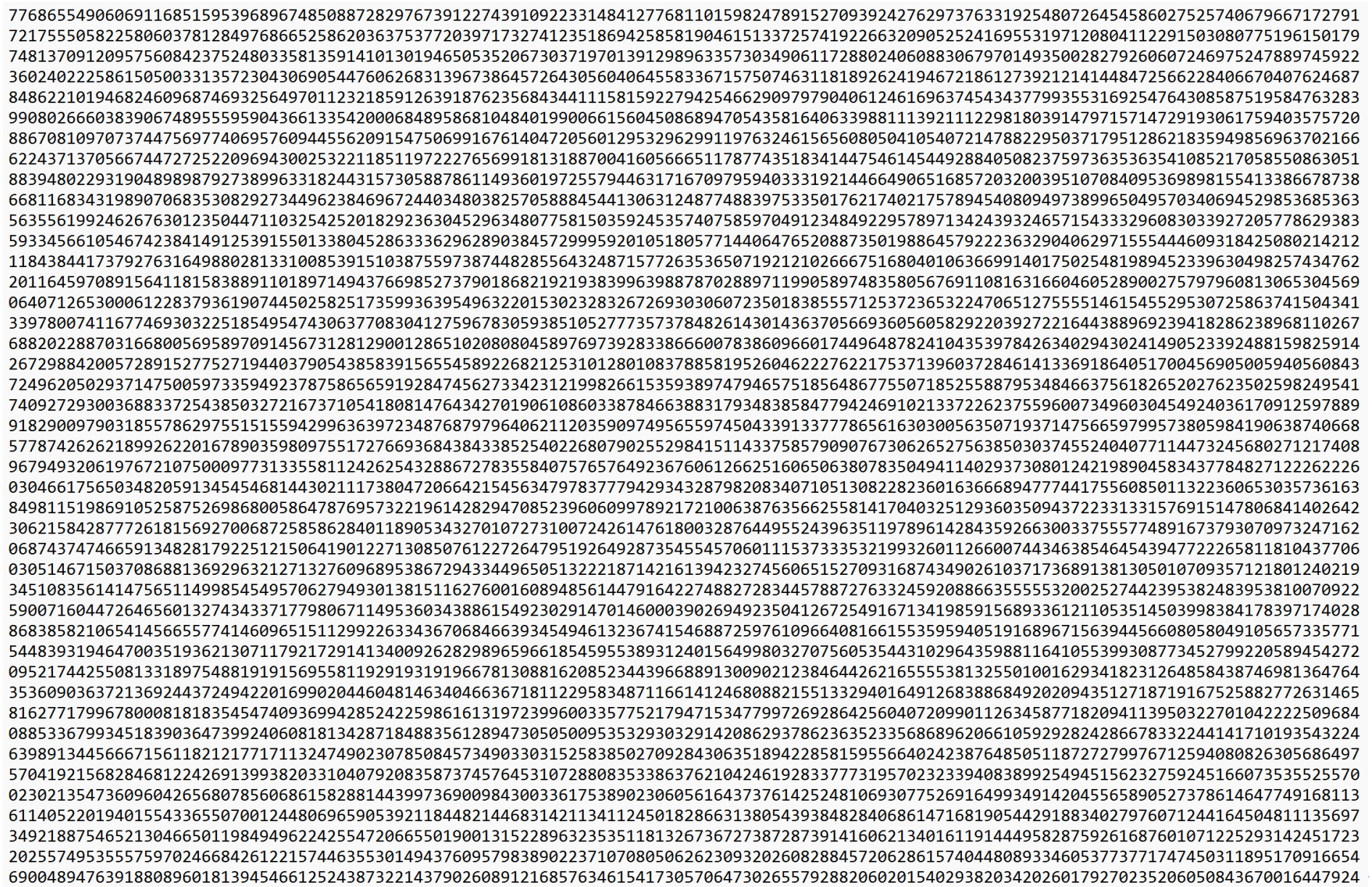
Again!

We will factor $N =$



Running the algorithm...

$a =$



Results

a has period 2 again!

$N = \gcd(a+1, N) \cdot \gcd(a-1, N)$, as desired.

The Catch

Finding a requires knowing p and q in advance.

Theorem

We can always find an $a < N$ such that $a \equiv 1 \pmod{p}$ and $a \equiv -1 \pmod{q}$

This guarantees that N divides $a^2 - 1$, so a has period 2. If additionally $p, q \neq 2$, we have that p divides $a - 1$ but not $a + 1$ and q divides $a + 1$ but not $a - 1$, so $\gcd(a \pm 1, N)$ is always p or q.

Conclusion

This is not a serious proposal. *However*, it is effectively what every implementation of Shor's algorithm so far has done (including 15).

```
def factor():
    print("Three times seven")
```

Figure: O'Farrell's constant-time factorization algorithm, shown here for 21.

Always remember $PR \neq PR$, and even PR can be misleading.